رواد مصر الرقمية

وزارة الاتصالات
وتكنولوجيا المعلومات

Data Preparation for Exploration

Session 3

Data Definition Language (DDL)

1. CREATE command
2. ALTER command
3. DROP command
4. TRUNCATE command

CREATE Command

- The **CREATE** command is used to **create new database** objects such as tables, views, indexes, and databases.

- Syntax

```
CREATE TABLE table_name
(column1 DATA_TYPE [CONS_TYPE],
 column2 DATA_TYPE [CONS_TYPE],... )
```

CREATE Command

- Example

```
CREATE TABLE Students (
    ID NUMBER(15) PRIMARY KEY,
    First_Name VARCHAR2(50) NOT NULL,
    Last_Name VARCHAR2(50),
    Address VARCHAR2(50),
    City VARCHAR2(50),
    Country VARCHAR2(25),
    Birth_Date DATE
);
```

EXERCISE

EMPLOYEE

FNAME	MINIT	LNAME	SSN	BDATE	ADDRESS	SEX	SALARY	SUPERSSN	DNO

DEPARTMENT

DNAME	DNUMBER	MGRSSN	MGRSTARTDATE

DEPT_LOCATIONS

DNUMBER	DLOCATION

PROJECT

PNAME	PNUMBER	PLOCATION	DNUM

WORKS_ON

ESSN	PNO	HOURS

DEPENDENT

ESSN	DEPENDENT_NAME	SEX	BDATE	RELATIONSHIP

Figure 7.5 Schema diagram for the COMPANY relational database schema; the primary keys are underlined.

ALTER Command

- The **ALTER** command is used to **modify** an existing database object. You can **add**, **delete**, or **modify** columns in a table, or **change** the table's properties.

- Syntax

```
ALTER TABLE table_name
ADD column_name datatype;
```

```
ALTER TABLE table_name
DROP COLUMN column_name;
```

ALTER Example

LastName	FirstName	Address
Pettersen	Kari	Storgt 20

To add a column named "City" in the “Students" table:

```
ALTER TABLE Students ADD City varchar(30)
```

Result:

LastName	FirstName	Address	City
Pettersen	Kari	Storgt 20	

DROP Command

- Syntax

```
DROP TABLE table_name;
```

DROP Command

- Example

```
DROP TABLE Students;
```

TRUNCATE Command

The TRUNCATE command is used to remove all rows from a table without deleting the table itself. It’s faster than DELETE because it doesn’t log individual row deletions.

- Syntax

```
TRUNCATE TABLE table_name
```

- Example

```
TRUNCATE TABLE employees;
```

Data Manipulation Language (DML)

1. INSERT Command
2. UPDATE Command
3. DELETE Command
4. SELECT Command

INSERT Command

Definition: The **INSERT** command is used to **add new rows of data** into a table.

Syntax:

```
INSERT INTO table_name (column1, column2, ...)
VALUES (value1, value2, ...);
```

Example:

```
INSERT INTO Store_Information (store_name, Sales, Date)
VALUES ('Los Angeles', 900, '1999-01-10');
```

Table Store_Information

Column Name	Data Type
store_name	char(50)
Sales	float
Date	datetime

INSERT Example 2

LastName	FirstName	Address	City
El-Sayed	Mohamed	Nasr City	Cairo

```
INSERT INTO Students  VALUES ('Saleh', 'Ahmed', 'Moharam bak', 'Alex.')
```

Result

LastName	FirstName	Address	City
El-Sayed	Mohamed	Nasr City	Cairo
Saleh	Ahmed	Moharam bak	Alex.

INSERT Example 3

LastName	FirstName	Address	City
El-Sayed	Mohamed	Nasr City	Cairo
Saleh	Ahmed	Moharam bak	Alex.

```
INSERT INTO Students (LastName, City) VALUES ('Hassan', 'Assuit')
```

Result

LastName	FirstName	Address	City
El-Sayed	Mohamed	Nasr City	Cairo
Saleh	Ahmed	Moharam bak	Alex.
Hassan			Assuit

UPDATE Command

Definition: The UPDATE command is used to **modify existing records** in a table.

Syntax

```
UPDATE table_name
SET column1 = value1, column2 = value2, ...
WHERE condition;
```

Example

```
UPDATE Students
SET City = 'Los Angeles'
WHERE ID = 101;
```

UPDATE Example

```
UPDATE Store_Information
SET Sales = 500
WHERE store_name = 'Los Angeles'
AND Date = 'Jan-08-1999'
```

Before

store_name	Sales	Date
Los Angeles	$1500	Jan-05-1999
San Diego	$250	Jan-07-1999
Los Angeles	$300	Jan-08-1999
Boston	$700	Jan-08-1999

After

store_name	Sales	Date
Los Angeles	$1500	Jan-05-1999
San Diego	$250	Jan-07-1999
Los Angeles	$500	Jan-08-1999
Boston	$700	Jan-08-1999

UPDATE Example 2

LastName	FirstName	Address	City
El-Sayed	Mohamed	Nasr City	Cairo
Saleh	Ahmed	Moharam bak	Alex.

```
UPDATE Students
SET Address = '241 El-haram', City = 'Giza'
WHERE LastName = 'El-Sayed';
```

Result:

LastName	FirstName	Address	City
El-Sayed	Mohamed	241 El-haram	Giza
Saleh	Ahmed	Moharam bak	Alex.

DELETE Command

- **Definition**: The **DELETE** command is used to **remove existing records** from a table.

- Syntax

```
DELETE FROM table_name
WHERE condition;
```

- Example

```
DELETE FROM Students
WHERE ID = 101;
```

DELETE Example

```
DELETE FROM  Store_Information
WHERE store_name = 'Los Angeles'
```

Before

store_name	Sales	Date
Los Angeles	$1500	Jan-05-1999
San Diego	$250	Jan-07-1999
Los Angeles	$300	Jan-08-1999
Boston	$700	Jan-08-1999

After

store_name	Sales	Date
San Diego	$250	Jan-07-1999
Boston	$700	Jan-08-1999

TRUNCATE vs DELETE

- **TRUNCATE TABLE** is functionally identical to DELETE statement with no WHERE clause
- Use **DELETE** when you need to remove specific rows and possibly roll back the operation.
- Use **TRUNCATE** when you want to quickly remove all rows from a table and reset it, without the need for rollback or triggers.

SELECT Statement

- The **SELECT** statement is one of the most fundamental and widely used SQL commands, allowing you to **retrieve** data from a **database**. It can be used to query data from **one** or **more** tables and can include various clauses to **filter**, **group**, and **sort** the data.

```
SELECT column1, column2, ...
FROM table_name
WHERE condition
GROUP BY column
HAVING condition
ORDER BY column [ASC|DESC];
```

SELECT Examples

```sql
SELECT *
    FROM departments;

SELECT emp_id, emp_name, dept_id
    FROM employees;

SELECT dept_id, dept_name
    FROM departments
    WHERE location = 'Cairo';
```

DISTINCT Keyword

- DISTINCT Keyword in SQL is used to remove duplicate rows from the result set, returning only unique values in the specified columns.

```
SELECT DISTINCT DNo
    FROM employees;
```

Employees table

EmpNo	Name	DNo	JobID
100	Ahmed	2	Sales_Rep
200	Mai	2	IT_PROG
300	Ali	2	Sales_Rep
400	Mahmoud	3	Sales_Rep

Output

DNo
2
3

DISTINCT Keyword (cont.)

```
SELECT DISTINCT DNo,JobID
    FROM employees;
```

Employees table

EmpNo	Name	DNo	JobID
100	Ahmed	2	Sales_Rep
200	Mai	2	IT_PROG
300	Ali	2	Sales_Rep
400	Mahmoud	3	Sales_Rep

Output

Dno	JobID
2	Sales_Rep
2	IT_PROG
3	Sales_Rep

SQL Aliases

- SQL aliases are used to give a table, or a column in a table, a temporary name
- Aliases are often used to make column names more readable.
- You can use `AS` to explicitly define an alias, or simply place the alias name directly after the column name for a more concise syntax.

```
SELECT CustomerID ID
FROM Customers;
```

```
SELECT CustomerID AS ID
FROM Customers;
```

Comparison Conditions

- = Equal
- > greater than
- >= greater than or equal
- < less than
- <= less than or equal
- <>not equal

```
SELECT last_name, salary
FROM employees
WHERE salary > 1000
```

Other Comparison Conditions

- Between …… and ….. (between two values inclusive)
- In (set) (match any of a list of values)
- All (set) (match all values in the list)
- Like (match a character pattern) (_ under score stands for any single character , % percent stands for any sequence of n character

LIKE Keyword

The `LIKE` keyword in SQL is used to search for a specified pattern in a column. It is often used with wildcard characters to find specific patterns within text data.

Wildcard	Explanation
%	Allows you to match any string of any length (including zero length)
_	Allows you to match on a single character

- A% means starts with A like ABC or ABCDE
- %A means anything that ends with A
- A%B means starts with A but ends with B
- AB_C means string starts with AB, then there is one character, then there is C

Examples

```
SELECT last_name, salary
FROM employees
WHERE salary BETWEEN 1000 AND 3000;
```

‘Between .. AND .. ’: This query retrieves the last_name and salary of employees whose salary is between 1000 and 3000, inclusive.

```
SELECT emp_id, last_name, salary, manager_id
FROM employees
WHERE manager_id IN (100, 101, 200);
```

IN (set) : This query retrieves the emp_id, last_name, salary, and manager_id of employees who have a manager_id of either 100, 101, or 200.

```
SELECT first_name
FROM employees
WHERE first_name LIKE '_s%';
```

LIKE KEYWORD: This query retrieves the first_name of employees where the name starts with any single character, followed by 's', and then any sequence of characters.

Logical Conditions

- AND
- OR
- NOT

```
SELECT emp_id, last_name, salary, manager_id
FROM employees
WHERE manager_id NOT IN (100, 101, 200)
AND salary > 1000;
```

Arithmetic Expressions

```
SELECT last_name, salary, salary + 300
   FROM employees;
```

Order of precedence: * , / , +, -

You can enforce priority by adding parentheses

```
SELECT last_name, salary, 10 * (salary + 300) AS 'Proposed salary'
   FROM employees;
```

Order by Clause (ASC, DESC)

```
SELECT fname, dept_id, hire_date
    FROM employees
    ORDER BY hire_date  DESC;
```

```
SELECT fname, dept_id, salary
    FROM employees
    ORDER BY dept_id ASC, Salary  DESC;
```

Practical SQL Challenges

- **Challenge 1:**Retrieve the top 5 employees with the highest sales in each department.

- **Challenge 2:**Identify products that have not been sold in the last 6 months.

Exercise #1

Column Name	Type
product_id	int
low_fats	enum
recyclable	enum

product_id is the primary key (column with unique values) for this table.
low_fats is an ENUM (category) of type ('Y', 'N') where 'Y' means this product is low fat and 'N' means it is not.
recyclable is an ENUM (category) of types ('Y', 'N') where 'Y' means this product is recyclable and 'N' means it is not.

Example

Input:
Products table:

product_id	low_fats	recyclable
0	Y	N
1	Y	Y
2	N	Y
3	Y	Y
4	N	N

Output:

product_id
1
3

Explanation: Only products 1 and 3 are both low fat and recyclable.

Exercise #2


```
+-------------+---------+
| Column Name | Type    |
+-------------+---------+
| id          | int     |
| name        | varchar |
| referee_id  | int     |
+-------------+---------+
In SQL, id is the primary key column for this table.
Each row of this table indicates the id of a customer, their name, and the id of the customer who referred them.
```

Find the names of the customer that are **not referred by** the customer with `id = 2`.

Return the result table in **any order**.

The result format is in the following example.

Example

Input:

Customer table:

id	name	referee_id
1	Will	null
2	Jane	null
3	Alex	2
4	Bill	null
5	Zack	1
6	Mark	2

Output:

name
Will
Jane
Bill
Zack

Exercise #3

```
+-------------+---------+
| Column Name | Type    |
+-------------+---------+
| name        | varchar |
| continent   | varchar |
| area        | int     |
| population  | int     |
| gdp         | bigint  |
+-------------+---------+
```

name is the primary key (column with unique values) for this table.
Each row of this table gives information about the name of a country, the continent to which it belongs, its area, the population, and its GDP value.

A country is **big** if:

- it has an area of at least three million (i.e., `3000000 km²`), or
- it has a population of at least twenty-five million (i.e., `25000000`).

Write a solution to find the name, population, and area of the **big countries**.

Return the result table in **any order**.

The result format is in the following example.

Example

Input:

World table:

name	continent	area	population	gdp
Afghanistan	Asia	652230	25500100	20343000000
Albania	Europe	28748	2831741	12960000000
Algeria	Africa	2381741	37100000	188681000000
Andorra	Europe	468	78115	3712000000
Angola	Africa	1246700	20609294	100990000000

Output:

name	population	area
Afghanistan	25500100	652230
Algeria	37100000	2381741

Exercise #4

Column Name	Type
article_id	int
author_id	int
viewer_id	int
view_date	date

There is no primary key (column with unique values) for this table, the table may have duplicate rows.
Each row of this table indicates that some viewer viewed an article (written by some author) on some date.
Note that equal author_id and viewer_id indicate the same person.

Write a solution to find all the authors that viewed at least one of their own articles.

Return the result table sorted by `id` in ascending order.

The result format is in the following example.

Example

Input:

Views table:

article_id	author_id	viewer_id	view_date
1	3	5	2019-08-01
1	3	6	2019-08-02
2	7	7	2019-08-01
2	7	6	2019-08-02
4	7	1	2019-07-22
3	4	4	2019-07-21
3	4	4	2019-07-21

Output:

id
4
7

Any Questions ?

Thank You